

Projet « EXPRESSO »

Architecture interne des systèmes d'exploitation

ISTY/M1-CHPS

2025/2026

1 Présentation

Les fils d'exécution sont une des abstractions du système d'exploitation. Ils permettent à une application de tirer l'avantage du parallélisme d'une architecture en mémoire partagée en répartissant la charge de travail aux différents cœurs du processeur. Les systèmes d'exploitation fournissent des interfaces de programmation de bas niveau pour créer et manipuler des fils d'exécution telles que les POSIX *threads*, ou **pthread**s. Malheureusement, le développement d'une application à base de **pthread**s, aussi simple soit-elle, requiert un effort de programmation important.

Des interfaces de haut niveau, telles qu'OpenMP [1], ont été développées dans le but de faciliter la conception d'applications parallèles en simplifiant certains aspects techniques liés à la parallélisation. Dans le cas d'OpenMP, un ensemble de directives de compilation permettent au programmeur d'exprimer différents types de parallélisme (parallélisme de données, parallélisme de tâches, ...) à travers des paradigmes de programmation tels que le *fork-join* ou le graphe de tâches sans manipulation directe de fils d'exécution.

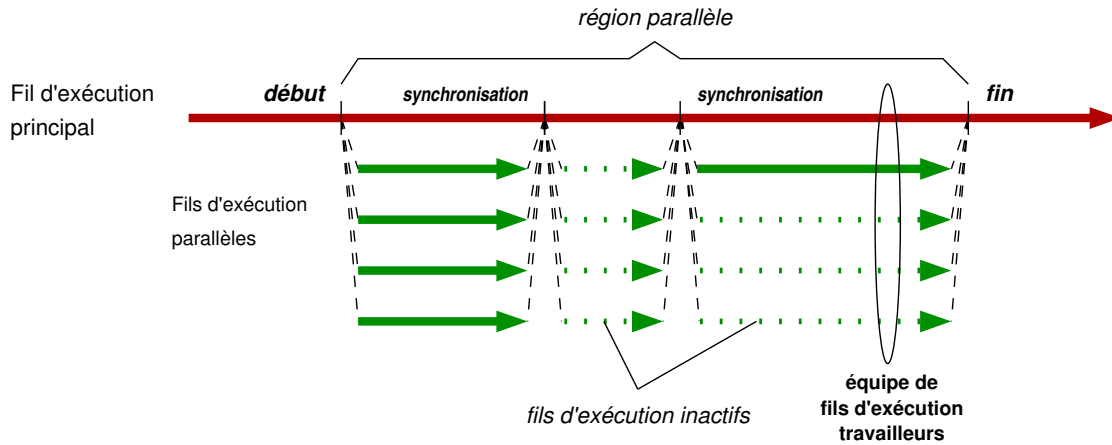
Pour vous permettre de mesurer la complexité de la conception d'une interface de programmation telle qu'OpenMP ainsi que pour bien comprendre les mécanismes sous-jacents, le but de ce projet est de développer une interface de programmation parallèle de haut niveau baptisée « EXPRESSO ».

2 EXPRESSO

EXPRESSO (= *Experimental parallel resource scheduler and organizer*) est une interface de programmation parallèle de haut niveau en mémoire partagée basée sur le paradigme de programmation *fork-join*. Dans un programme, elle permet de mettre en place une région parallèle contenant des tâches parallélisables et un ou plusieurs points de synchronisation.

Une tâche parallélisable est définie par une suite d'instructions qui peuvent être exécutées en parallèle à une autre suite d'instructions du programme. Puis, à chaque tâche parallélisable, il est possible d'associer un ensemble de données nécessaires pour son exécution, c'est-à-dire les données d'entrée et de sortie de la tâche.

Dans ce but, EXPRESSO implémente un moteur d'exécution constitué d'un ordonnanceur et d'une équipe de fils d'exécution travailleurs qui inclut le fil d'exécution principal du programme et un certain nombre de fils d'exécution parallèles. La taille de cette équipe est fixée en début de la région parallèle et reste inchangée jusqu'à la fin de celle-ci. Par conséquent, lorsqu'à un moment donné il n'y a pas suffisamment de tâches parallélisables pour occuper tous les fils d'exécution travailleurs, certains restent inactifs. Le schéma ci-dessous illustre l'exécution d'un programme parallèle à base de EXPRESSO.



Les **tâches parallélisables** sont définies et envoyées au moteur d'exécution pour être **exécutées de façon asynchrone**. Autrement dit, l'exécution effective des tâches ne commence qu'au premier point de synchronisation rencontré. Elle se termine avec le dernier fil d'exécution travailleur qui franchi le point de synchronisation. Dès lors, le moteur d'exécution redevient prêt à recevoir de nouvelles tâches ou à sortir de la région parallèle.

À la différence d'OpenMP, EXPRESSO ne définit pas de directives de compilation, mais repose exclusivement sur des appels à fonctions. C'est une bibliothèque écrite en langage C.

2.1 Version de base

Dans sa version de base, la bibliothèque EXPRESSO expose l'interface de programmation (API) suivante :

- `int espresso_initialize();` – lecture de paramètres, allocations de structures de données internes, création et lancement de fils d'exécution parallèles, ...
- `size_t espresso_worker_count();` – récupération du nombre de fils d'exécution travailleurs de l'équipe (y compris le fil d'exécution principal),
- `int espresso_task(void (*) (void *), void *);` – envoi d'une tâche parallélisable au moteur d'exécution (un pointeur vers une fonction qui contient les instructions à effectuer et un pointeur vers une variable ou une structure passée en argument de cette fonction qui représente les données d'entrée et de sortie de la tâche),
- `int espresso_wait();` – lancement de l'exécution des tâches parallélisables précédemment envoyées au moteur d'exécution avec la fonction `espresso_task`,
- `void espresso_stats();` – affichage des statistiques sur le nombre de tâches exécutées par chaque fil d'exécution travailleur, le nombre total de tâches exécutées par tous les fils d'exécution travailleurs et la moyenne du nombre de tâches exécutées par chaque fil d'exécution travailleur,
- `double espresso_wall_time();` – récupération d'un repère temporel (nombre de secondes écoulées depuis un temps de référence),
- `int espresso_finalize();` – terminaison des fils d'exécution parallèles, libération de la mémoire allouée pour les structures de données internes, nettoyage, ...

Par défaut, la fonction `espresso_initialize` crée une équipe avec autant de fils d'exécution travailleurs qu'il y a de cœurs logiques sur la machine qui fait tourner le programme parallèle. Il est possible de contrôler le nombre de fils d'exécution travailleurs à l'aide de la variable d'environnement `EXPRESSO_WORKER_COUNT`. Par exemple,

pour exécuter un programme nommé `exemple` avec deux fils d'exécution travailleurs, nous pouvons utiliser la commande suivante.

```
EXPRESSO_WORKER_COUNT=2 ./exemple
```

Il est tout-à-fait possible de positionner `EXPRESSO_WORKER_COUNT` à 1 pour n'utiliser qu'un seul fil d'exécution travailleur, c'est-à-dire le fil d'exécution principal.

À l'exception du fil d'exécution principal, les autres fils d'exécution de l'équipe restent inactifs jusqu'au premier appel à `expresso_wait` qui déclenche l'exécution des tâches envoyées au moteur d'exécution à l'aide de la fonction `expresso_task`.

L'ordonnancement des tâches parallélisables se fait suivant une politique d'ordonnancement dynamique. Lorsqu'une tâche est envoyée au moteur d'exécution avec `expresso_task`, elle est placée dans une file d'attente. Une fois l'exécution déclenchée avec `expresso_wait`, les fils d'exécution travailleurs viennent chercher une tâche à exécuter dans la file d'attente dès qu'ils sont prêts. Ce processus se répète jusqu'à ce qu'il n'y ait plus de tâches dans la file d'attente. À ce moment-là, les fils d'exécution travailleurs, sauf le fil d'exécution principal, redeviennent inactifs jusqu'à un nouvel appel à `expresso_wait` ou `expresso_finalize`.

L'exemple suivant illustre l'utilisation de cette API en pratique.

2.1.1 Exemple d'utilisation

Ci-dessous à gauche se trouve l'implémentation séquentielle d'un programme qui calcule la somme `sum` de quatre entiers `a`, `b`, `c` et `d` avant d'afficher le résultat sur sa sortie standard. Les sommes intermédiaires `a + b` et `c + d` peuvent être effectuées en parallèle et représentent donc deux tâches parallélisables.

Ci-dessous au milieu se trouve le même programme parallélisé avec EXPRESSO. La fonction `basic_sum_task` définit les instructions à effectuer par une tâche, c'est-à-dire une somme. La structure de données `s_data` représente le moyen de transmettre les données de calcul nécessaires à chaque tâche, c'est-à-dire les opérandes de la somme, ainsi que de récupérer le résultat dans une des opérandes. Notez que le choix de la structure de données est arbitraire et ne dépend que de l'utilisateur de la bibliothèque EXPRESSO.

Enfin, ci-dessous à droite se trouve une autre implémentation parallèle du même programme à base de EXPRESSO qui montre l'utilisation de plus d'un point de synchronisation. Dans cette version, la somme finale est calculée dans le cadre d'une troisième tâche qui s'exécute en séquentiel une fois que les deux premières tâches ont terminé leur exécution.

```

#include <stdio.h>
#include "expresso.h"

struct s_data { int a, b; };
void basic_sum_task(void * data) {
    struct s_data * this =
        (struct s_data *) data;
    this->a += this->b;
}
int main(int argc, char ** argv) {
    int a = 10, b = 11, c = 7, d = 14;
    int sum;
    struct s_data * data =
        malloc(2 * sizeof(struct
            ↪ s_data));
    expresso_initialize();
    data[0].a = a;
    data[0].b = b;
    expresso_task(
        &basic_sum_task,
        (void *) &data[0]
    );
    data[1].a = c;
    data[1].b = d;
    expresso_task(
        &basic_sum_task,
        (void *) &data[1]
    );
    expresso_wait();
    sum = data[0].a + data[1].a;
    expresso_stats();
    expresso_finalize();
    printf("%d\n", sum);
    free(data);
    return 0;
}

#include <stdio.h>
#include "expresso.h"

struct s_data { int a, b; };
void basic_sum_task(void * data) {
    struct s_data * this =
        (struct s_data *) data;
    this->a += this->b;
}
int main(int argc, char ** argv) {
    int a = 10, b = 11, c = 7, d = 14;
    int sum;
    struct s_data * data =
        malloc(2 * sizeof(struct
            ↪ s_data));
    expresso_initialize();
    data[0].a = a;
    data[0].b = b;
    expresso_task(
        &basic_sum_task,
        (void *) &data[0]
    );
    data[1].a = c;
    data[1].b = d;
    expresso_task(
        &basic_sum_task,
        (void *) &data[1]
    );
    expresso_wait();
    data[0].b = data[1].a;
    expresso_task(
        &basic_sum_task,
        (void *) &data[0]
    );
    expresso_wait();
    sum = data[0].a;
    expresso_stats();
    expresso_finalize();
    printf("%d\n", sum);
    free(data);
    return 0;
}

```

2.2 Version avancée

Dans la version avancée, l'API de la bibliothèque EXPRESSO est enrichie des éléments suivants :

- un type `schedule_t` et des fonctions permettant de choisir une politique d'ordonnancement ou de récupérer la politique d'ordonnancement actuellement en vigueur,

```

typedef enum {
    SCHEDULE_DYNAMIC,
    SCHEDULE_STATIC,
    SCHEDULE_BALANCED
} schedule_t;
schedule_t expresso_schedule_get();
void expresso_schedule_set(schedule_t);

```

- une fonction de définition et d'envoi de tâches permettant d'associer un poids (un entier non-signé strictement positif) à chaque tâche.

```

int expresso_weighted_task(void (*) (void *), void *, unsigned int);

```

L'objectif est de donner au programmeur le moyen de contrôler la façon dont les tâches parallélisables sont ordonnancées. La politique d'ordonnancement dynamique (`SCHEDULE_DYNAMIC` dans `schedule_t`) est la politique utilisée par la version de base de la bibliothèque (voir la section 2.1). Les politiques d'ordonnancement statique (`SCHEDULE_STATIC` dans `schedule_t`) et équilibrée (`SCHEDULE_BALANCED` dans `schedule_t`) répartissent les tâches aux fils d'exécution travailleurs avant le début de l'exécution des tâches.

La politique statique vise à répartir les tâches aux fils d'exécution travailleurs dans l'ordre : la première tâche envoyée sera attribuée au fil d'exécution numéro 0, la deuxième tâche au fil d'exécution numéro 1 et ainsi de suite. S'il y a plus de tâches que de fils d'exécution, l'attribution reprend à partir du fil d'exécution numéro 0.

La politique équilibrée vise à prendre en compte une potentielle différence de la charge de travail entre les tâches. La fonction `expresso_weighted_task` permet d'associer un poids à chaque tâche envoyée au moteur d'exécution. En se basant sur le poids des tâches, la politique équilibrée tente d'attribuer les tâches aux fils d'exécution travailleurs de façon à ce que la charge de travail soit équitablement répartie.

Par défaut, la version avancée utilise la politique d'ordonnancement dynamique. Il est possible de changer de politique d'ordonnancement grâce à la variable d'environnement `EXPRESSO_SCHEDULE` qui accepte les valeurs `dynamic`, `static` ou `balanced`. Puis, la fonction `expresso_schedule_set` permet de changer de politique d'ordonnancement depuis la région parallèle avant un appel à `expresso_wait`.

3 Réalisation

Ce projet a deux objectifs :

1. implémenter la bibliothèque EXPRESSO,
2. utiliser EXPRESSO pour paralléliser une collection d'applications séquentielles.

Le point de départ pour la réalisation de ce projet sont les dépôts git suivants :

- `expresso` à l'adresse <https://gitlab.com/expresso-student/expresso>
 - squelette d'implémentation de la bibliothèque EXPRESSO à compléter
- `expresso-benchmarks` à l'adresse <https://gitlab.com/expresso-student/expresso-benchmarks>
 - applications à paralléliser à l'aide de la bibliothèque EXPRESSO

3.1 Dépôt expresso

Voici la structure de fichiers du dépôt.

```
expresso
|-- tests
|   |-- .gitignore
|   |-- Makefile
|   |-- add-to-all.c
|   |-- add-to-all-twice.c
```

```
|    |-- basic-sum.c
|    |-- no-tasks.c
|-- .gitignore
|-- Makefile
|-- espresso.h
```

Le fichier en-tête **espresso.h** définit l'API de EXPRESSO telle qu'elle est décrite dans les sections 2.1 et 2.2. Le fichier source correspondant, c'est-à-dire **espresso.c**, contient les coquilles vides des définitions des fonctions déclarées dans **espresso.h**.

Dans le dossier **tests**, vous trouverez un ensemble de programmes composés d'une implémentation séquentielle de référence et d'une implémentation parallèle utilisant la bibliothèque EXPRESSO. Ces programmes, qui vous sont fournis clés en main, vous permettront de tester votre implémentation tout au long du cycle de développement.

Puis, il y a deux recettes de construction **Makefile**.

Le **Makefile** à la racine du dépôt permet de :

- compiler le fichier source **espresso.c** en un fichier binaire **espresso.o**,
- construire EXPRESSO en tant que bibliothèque statique **libespresso.a**, mais aussi en tant que bibliothèque dynamique **libespresso.so** (cible par défaut **all**),
- installer/désinstaller EXPRESSO (cibles **install** et **uninstall**) à l'emplacement indiqué par la variable **PREFIX** (voir **Makefile**),
- compiler et exécuter l'ensemble des tests dans le dossier **tests** (cible **test**),
- nettoyer les fichiers produits lors de la compilation (cible **clean**).

Le **Makefile** dans le dossier **tests** sert à :

- construire tous les programmes de test (cible par défaut **all**) et les lier à la version statique de la bibliothèque **libespresso.a**,
- exécuter tous les programmes de test pour différentes configurations (nombre de fils d'exécution travailleurs, politique d'ordonnancement) de la bibliothèque EXPRESSO (cible **run**),
- nettoyer les fichiers produits lors de la compilation des programmes de test (cible **clean**).

Enfin, à l'aide des commandes suivantes, vous pouvez obtenir une copie locale du dépôt **espresso**, construire la bibliothèque EXPRESSO, puis compiler et exécuter les programmes de test.

```
git clone https://gitlab.com/espresso-student/espresso.git
cd espresso
make
make test
```

La commande ci-dessous (exécutée depuis la racine du dépôt **espresso**) permet d'installer EXPRESSO dans un répertoire spécifique à l'emplacement désigné par la variable **PREFIX**.

```
PREFIX=$HOME/bin/espresso make install
```

3.2 Dépôt `expresso-benchmarks`

Voici la structure de fichiers du dépôt.

```
expresso-benchmarks
|-- .gitignore
|-- Makefile
|-- dot.c
|-- list.c
|-- matmul.c
|-- vecmul.c
```

Chaque fichier source `.c` contient les implémentations séquentielles de quatre applications. Les commentaires dans les fichiers sources vous aideront à comprendre le fonctionnement de celles-ci.

La recette `Makefile` sert à compiler les applications (cible par défaut `all`) en les liant à la version partagée de la bibliothèque `EXPRESSO` préalablement installée à l'emplacement indiqué par la variable `PREFIX` (voir `Makefile`). La commande `make test` permet d'exécuter l'ensemble des applications pour différentes configurations (nombre de fils d'exécution travailleurs, politique d'ordonnancement) de `EXPRESSO`.

À l'aide des commandes suivantes, vous pouvez obtenir une copie locale du dépôt `expresso-benchmarks`, compiler et exécuter les quatre applications. En positionnant la variable `PREFIX`, nous indiquons au compilateur où trouver une installation de `EXPRESSO`.

```
git clone https://gitlab.com/expresso-student/expresso.git
cd expresso
PREFIX=$HOME/bin/expresso make
PREFIX=$HOME/bin/expresso make test
```

3.3 Objectif 1 : Implémentation de la bibliothèque `EXPRESSO`

Vous réaliserez l'objectif 1 à partir du squelette de l'implémentation dans le dépôt `expresso`. Votre rôle sera d'implémenter les fonctions de l'API de la bibliothèque `EXPRESSO` déclarées dans `expresso.h` et définies dans `expresso.c`.

3.3.1 Conseils et contraintes d'implémentation

Procédez par étapes. Dans un premier temps, implémenter seulement les fonctionnalités de la version de base de `EXPRESSO` (voir la section 2.1). Mettez continuellement votre implémentation à l'épreuve grâce aux programmes de test fournis. N'hésitez pas à en implémenter d'autres. Le fichier `Makefile` dans le dossier `tests` contient des commentaires expliquant comment inclure de nouveaux programmes test à la recette de construction. Si (et seulement si) votre implémentation de la version de base fonctionne correctement, passez à l'implémentation de la version avancée (voir la section 2.2).

Pour plus de clarté et de maintenabilité, vous pouvez structurer votre implémentation en plusieurs fichiers sources au-delà du seul fichier `expresso.c`. Vous pourrez alors facilement les inclure à la recette de construction `Makefile` à la racine du dépôt, en modifiant uniquement la ligne 9 selon les instructions dans ce fichier.

Ne modifiez ni le fichier `expresso.h` ni les programmes de test fournis.

3.4 Objectif 2 : Parallélisation à l'aide de la bibliothèque EXPRESSO

Étudiez les implémentations séquentielles des applications dans le dépôt `expresso-benchmarks` et parallélisez-les en faisant appel aux fonctions de l'API de la bibliothèque EXPRESSO. Pour compiler les applications et vérifier leur bon fonctionnement, vous vous appuyerez sur votre implémentation de EXPRESSO. Vous pourrez commencer vos tests dès que vous aurez terminé l'implémentation de la version de base de la bibliothèque.

Réalisez les implémentations parallèles directement dans les fichiers source existants à côté des implémentations séquentielles. Comme l'illustre l'exemple ci-dessous (adapté de l'exemple dans la section 2.1), pour chaque application, le but est de pouvoir :

- comparer le résultat calculé en séquentiel et le résultat calculé en parallèle afin de vérifier si l'implémentation parallèle est correcte,
- mesurer et afficher les temps d'exécution des deux implémentations.

```
#include <stdio.h>
#include "expresso.h"

struct s_data { int a, b; };

void basic_sum_task(void * data) {
    struct s_data * this = (struct s_data *) data;
    this->a += this->b;
}

int main(int argc, char ** argv) {
    int a = 10, b = 11, c = 7, d = 14;
    int sum_reference, sum_expresso;
    double time_reference = 0.0, time_expresso = 0.0;

    // Implémentation séquentielle.
    time_reference = expresso_wall_time();
    sum_reference = a + b + c + d;
    time_reference = expresso_wall_time() - time_reference;

    // Implémentation parallèle.
    struct s_data * data = malloc(2 * sizeof(struct s_data));
    expresso_initialize();

    time_expresso = expresso_wall_time();
    data[0].a = a;
    data[0].b = b;
    expresso_task(&basic_sum_task, (void *) &data[0]);
    data[1].a = c;
    data[1].b = d;
    expresso_task(&basic_sum_task, (void *) &data[1]);
    expresso_wait();
    data[0].b = data[1].a;
    expresso_task(&basic_sum_task, (void *) &data[0]);
    expresso_wait();
    sum_expresso = data[0].a;
    time_expresso = expresso_wall_time() - time_expresso;

    expresso_stats();
    expresso_finalize();

    // Affichage des résultats.
    printf("Reference result: %d\n", sum_reference);
    printf(" ExPResSO result: %d\n", sum_expresso);
```

```

// Vérification des résultats.
if(sum_reference != sum_expresso) {
    fprintf(stderr, "Bad results :-(\n");
    free(data);
    return 1;
}

// Affichage des temps d'exécution.
printf("Reference computation time: %g s\n", time_reference);
printf(" ExPResSO computation time: %g s\n", time_expresso);

printf("Good results :-(\n");
free(data);
return 0;
}

```

Les applications `dot` et `list` affichent toujours le résultat des calculs. Les applications `vecmul` et `matmul` ne le font que pour de petites tailles de problèmes pour ne pas saturer la sortie standard.

3.4.1 Conseils et contraintes d'implémentation

Dans un premier temps, étudiez bien le code source des applications et familiarisez-vous avec leur fonctionnement. Utilisez le `Makefile` présent dans le dépôt pour les compiler telles quelles, exécutez-les, testez différents paramètres de calculs.

Ensuite, procédez par étape. Tentez d'abord de paralléliser l'application qui vous semble la plus facile à paralléliser. Il est préférable de bien paralléliser une application que de mal paralléliser toutes les applications.

Ne modifiez ni le `Makefile` fourni, ni les implémentations séquentielles ni les fonctions d'initialisation, d'affichage, de vérification de résultats ou de nettoyage.

En mesurant le temps d'exécution des implémentations parallèles, ne prenez pas en compte l'initialisation, le paramétrage ou la terminaison de EXPRESSO, c'est-à-dire les appels à `expression_initialize`, `expresso_worker_count`, `expresso_stats`, `expresso_schedule_get`, `expresso_schedule_set` ou `expresso_finalize`). Ces étapes ne font pas partie du calcul.

3.5 Rapport

Vous rédigerez un rapport (entre 2 et 4 pages) dans lequel :

- vous décrierez et justifierez vos choix d'implémentation de la bibliothèque EXPRESSO,
- vous décrierez et justifierez vos choix algorithmiques en matière de parallélisation des applications du dépôt `expresso-benchmarks`,
- vous discuterez des performances de votre implémentation de EXPRESSO en vous basant sur les temps d'exécutions des applications parallélisées.
- vous discuterez de la différence entre les politiques d'ordonnancement que vous aurez implémentées dans EXPRESSO et de leur impact sur la performance des applications parallélisées.

4 Modalités de rendu

Votre implémentation est à réaliser à la fois dans votre copie du dépôt **expresso** et dans votre copie du dépôt **expresso-benchmarks**. Vous veillerez à la préservation des arborescences dans ces répertoires lors de la réalisation de votre projet. L'évaluateur devra pouvoir compiler la bibliothèque, les programmes de test et les applications du dépôt **expresso-benchmarks**, avec les fichiers **Makefile** d'origine prévus à cet effet.

Le projet est à rendre par courriel à l'adresse **marek.felsoci@uvsq.fr**. Le rendu se résume à une seule archive au format **.tar.gz** ou **.zip** nommée **NOM.Prenom.tar.gz** ou **NOM.Prenom.zip**. À l'extraction, cette archive devra produire :

- un répertoire **expresso** (contenant votre implémentation de la bibliothèque **EXPRESSO** et respectant l'arborescence d'origine),
- un répertoire **expresso-benchmarks** (contenant les applications parallélisées avec **EXPRESSO** et respectant l'arborescence d'origine),
- votre rapport au format **.pdf**.

Enfin, l'objet du courriel devra avoir la forme « [PROJET-AISE-2025] NOM Prénom ». La date limite de rendu est le **7 janvier 2026, minuit** (heure de Paris). Les rendus envoyés après cette date ne seront pas évalués !

5 Soutenance

Vous serez amenés à soutenir votre projet au cours d'une séance de questions d'une durée de **10 minutes** le **9 janvier 2026**. Les soutenances se dérouleront entre 9h30 et 17h00 dans la **salle Cartables numériques 1** du **bâtiment Rabelais**. Les horaires de passage individuelles vous seront communiquées par courriel.

6 Modalités d'évaluation

Le projet sera évalué sur la base de deux notes sur 20 points, une pour le rendu et une pour la soutenance. Le rendu sera évalué selon le barème approximatif suivant.

1. implémentation de la bibliothèque **EXPRESSO** (13 points)
 - version de base : 9 points
 - version avancée : 4 points
2. parallélisation d'applications à l'aide de **EXPRESSO** (5 points)
 - parallélisation de l'application **dot** : 1 point
 - parallélisation de l'application **list** : 1 point
 - parallélisation de l'application **matmul** : 2 points
 - parallélisation de l'application **vecmul** : 1 point
3. rapport (2 points)

References

- [1] OpenMP Architecture Review Board. *OpenMP Application Program Interface Version 6.0*. Nov. 2024. URL: <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-6-0.pdf>.